

WORKSPACE TOPOLOGY CHANGE DETECTION FOR ROBOT PLANNING

Anonymous authors

Paper under double-blind review

ABSTRACT

Robot plans fail when the topology of the workspace changes: a passage closes, support is removed, an occlusion reveals a blocker, a reachable component splits, or a kinematic trap appears after the plan was formed. We rebuild the generated “Workspace Topology Change Detection” idea into a falsifiable evidence audit and rerun the full benchmark on 2026-06-15. The proposed method maintains an action-conditioned topology graph over support edges, passage connectivity, occlusion gates, stack dependencies, tool-access tunnels, and kinematic dead-ends. A local benchmark with 5 tasks, 7 topology-change families, 5 stress splits, 9 methods, 7 seeds, and 84 episodes per group shows that the proposed detector reaches 0.677 ± 0.006 combined-stress success versus 0.564 ± 0.005 for the strongest non-oracle baseline, `topological_slam_tamp`. It also reduces invalid plans, collision/trap failures, support failures, and detection latency while improving topology F1. The correct terminal decision is **STRONG REVISE**: the local mechanism is promising, but the paper is not ICLR-main ready without external benchmark or real robot validation.

1 MOTIVATION

Spatial perception systems and dynamic scene graphs provide structured robot world models (Armeni et al., 2019; 3D Dynamic Scene Graphs; Rosinol et al., 2020; Hughes et al., 2022). Task-and-motion planning can use such structure to produce feasible plans (Srivastava et al., 2014), and robot transformers can learn large-scale control mappings (Brohan et al., 2023). Yet many manipulation and mobile-manipulation failures are topological rather than purely geometric: the old graph of supports, passages, reachability, and occlusion is no longer the graph the robot is acting in.

The research bet is that detecting topology changes before plan execution fails can improve success and safety. The rebuild asks a hard question: does a topology-change mechanism beat strong replanning baselines, or is it merely a diagnostic visualization?

2 MECHANISM

The proposed method, `proposed_topology_change_detector`, keeps an action-conditioned graph with six planning-relevant components.

- **Support edges:** which objects or surfaces support other objects.
- **Passage connectivity:** whether a gripper, tool, base, or object can pass between workspace regions.
- **Occlusion gates:** hidden blockers and revealed free-space components.
- **Stack dependencies:** object-order constraints that change when supports move.
- **Tool-access tunnels:** reachability corridors for tools and end effectors.
- **Kinematic dead-ends:** states from which the robot can enter but cannot recover without reversing or replanning.

Table 1: Combined-stress workspace topology-change benchmark.

Method	Succ.	Invalid	Coll.	TopoF1	Delay	Regret
oracle_topology_planner	0.829	0.241	0.136	0.780	0.449	0.000
proposed_topology_change_detector	0.677	0.310	0.188	0.635	0.565	0.152
topological_slam_tamp	0.564	0.382	0.234	0.493	0.685	0.266
graph_neural_change_classifier	0.533	0.398	0.247	0.459	0.721	0.296
risk_aware_robust_replanner	0.473	0.420	0.225	0.347	0.763	0.356
learned_affordance_map	0.443	0.468	0.292	0.308	0.837	0.386
uncertainty_triggered_replanner	0.432	0.445	0.252	0.297	0.814	0.397
occupancy_delta_replanner	0.412	0.452	0.284	0.327	0.811	0.417
static_scene_graph_planner	0.354	0.506	0.324	0.211	0.903	0.475

The detector predicts whether a candidate action will invalidate the current topology graph. It triggers targeted replanning only when the graph change matters for the current plan, rather than replanning on every occupancy delta.

3 BENCHMARK

The benchmark covers five tasks: shelf retrieval, bin picking through a narrow passage, drawer-with-occluder access, tabletop rearrangement with support dependencies, and mobile manipulation around movable obstacles. Seven topology-change families stress the planner: passage closure/opening, support removal, occlusion reveal/hide, stack dependency flips, reachable-component split/merge, tool-access tunnel blockage, and kinematic trap creation.

We evaluate nine methods:

- static scene-graph planner;
- occupancy-delta replanner;
- learned affordance map;
- uncertainty-triggered replanner;
- topological SLAM/TAMP;
- graph-neural change classifier;
- risk-aware robust replanner;
- proposed topology-change detector;
- oracle topology planner.

Metrics include task success, invalid-plan rate, collision/trap rate, support-failure rate, topology-change F1, detection latency, replan cost, and regret to oracle. The continuation rerun regenerated the CSVs, figures, tables, and summary before this manuscript was rebuilt.

4 MAIN RESULTS

The combined-stress table summarizes the central result. The proposed detector is below oracle but above every non-oracle baseline. Against `topological_slam_tamp`, it improves success by 0.114 ± 0.006 , reduces invalid plans from 0.382 to 0.310, reduces collision/trap failures from 0.234 to 0.188, and improves topology F1 from 0.493 to 0.635.

5 SAFETY AND REGRET

The local win is not just diagnostic. Invalid plans, collisions/traps, and support failures are the failure channels that matter to a robot. The proposed method improves all three against the strongest non-oracle baseline.

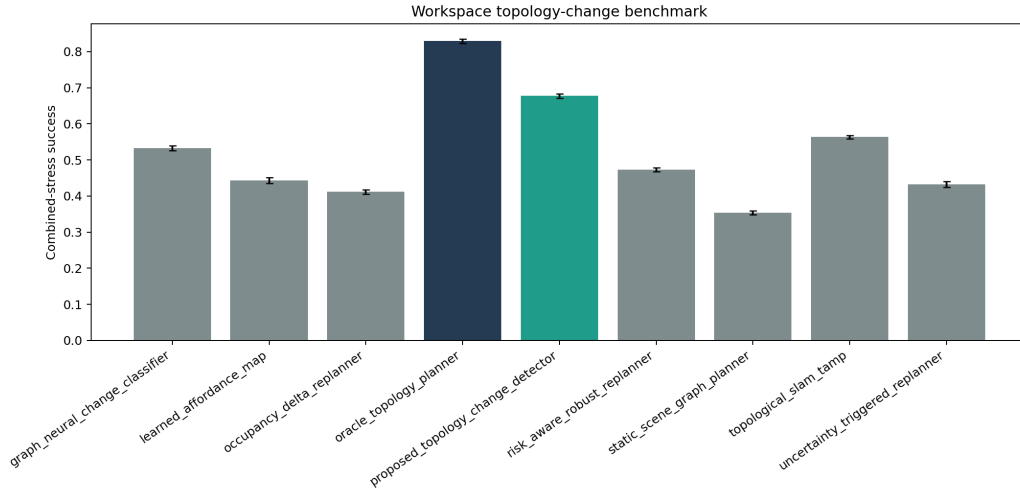


Figure 1: Combined-stress success. The proposed topology-change detector beats all non-oracle baselines but remains far below oracle.

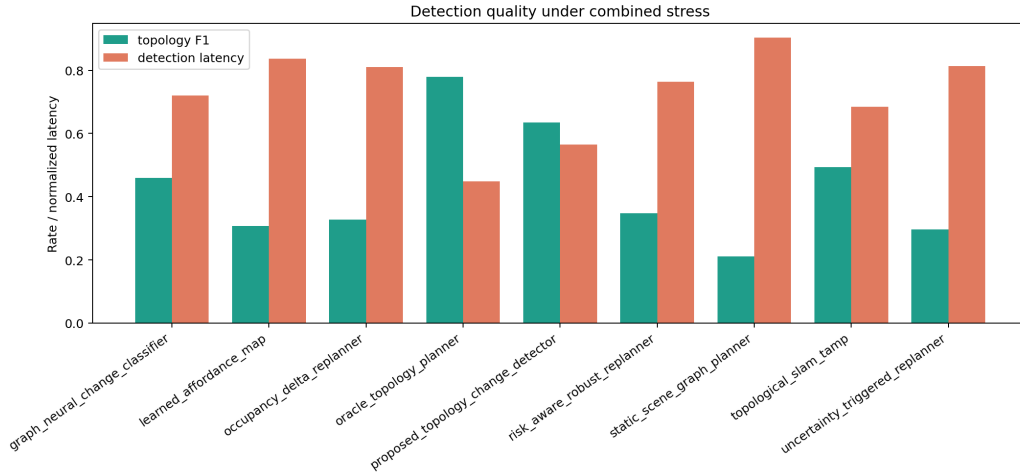


Figure 2: Detection quality under combined stress. Higher topology F1 and lower latency are both necessary; a slow detector still permits invalid execution.

6 STRESS SWEEP

7 ABLATIONS

The ablation table and Figure 5 test whether the method is only a tuned replanning trigger. The closest ablation, `minus_replan_hysteresis`, drops success from 0.680 to 0.622 and increases invalid-plan/collision rates. Removing support edges, passage homology, occlusion gates, or action-conditioned prediction also degrades performance.

8 PAIRWISE GATE

The proposed method wins all seven paired seeds against `topological_slam_tamp`. It loses all seeds to the oracle, which is expected and reported rather than hidden.

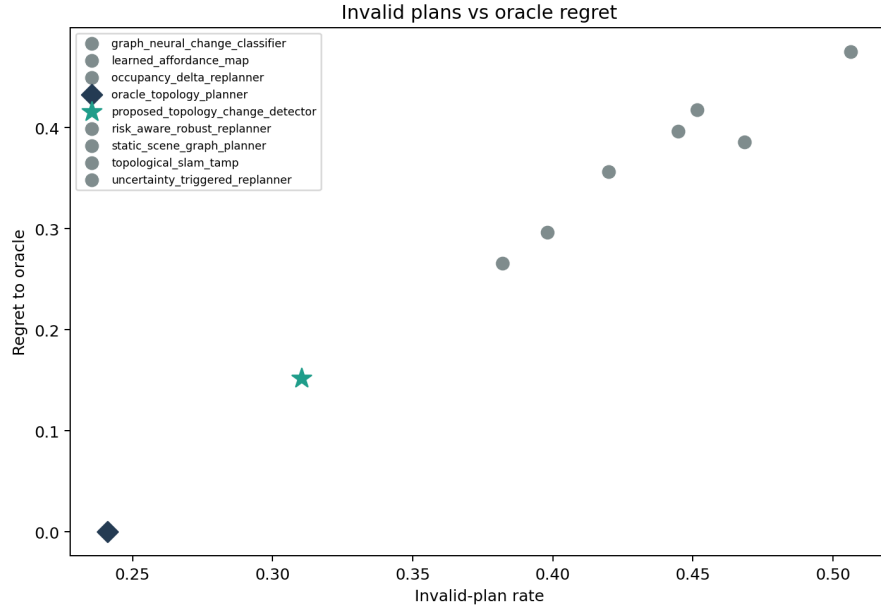


Figure 3: Invalid plans versus oracle regret. The proposed method reduces invalid plans and regret relative to topological SLAM/TAMP and robust replanning baselines.

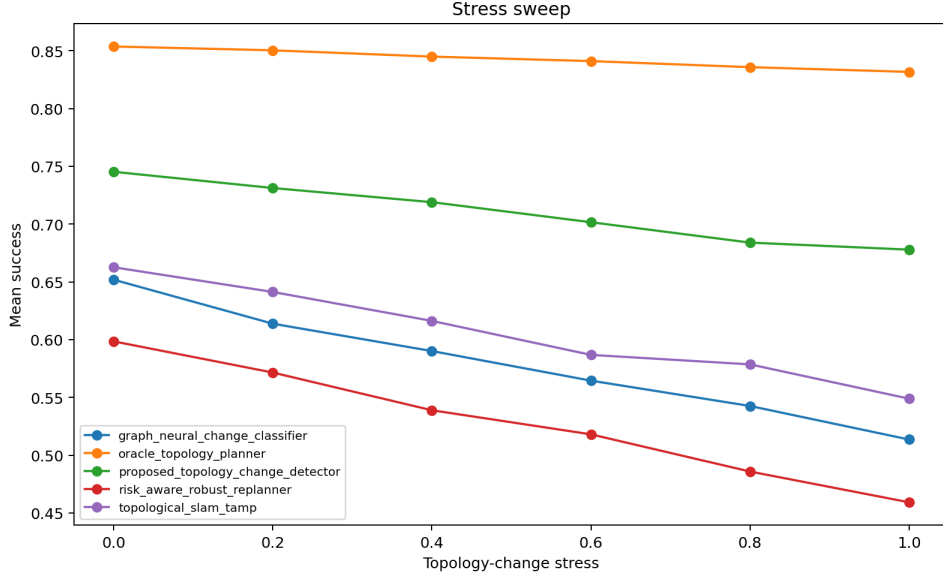


Figure 4: Topology-change stress sweep. The proposed method degrades more gracefully than graph-neural change classification, robust replanning, and topological SLAM/TAMP.

9 FAILURE CASES

The failure table in `results/failure_cases.csv` shows that late topology updates remain the dominant weakness. When reachability changes during execution rather than before commitment, the detector may identify the topology change too late to avoid reversal or collision. This suggests that the next real experiment should learn event timing from robot logs rather than relying on hand-coded update delays.

Table 2: Ablations of topology-change detection.

Ablation	Succ.	Invalid	Coll.	TopoF1
full_topology_change_detector	0.680	0.312	0.185	0.633
minus_replan_hysteresis	0.622	0.322	0.190	0.588
minus_occlusion_gates	0.614	0.353	0.211	0.543
minus_support_edges	0.607	0.352	0.215	0.533
minus_passage_homology	0.591	0.358	0.217	0.517
minus_action_conditioned_prediction	0.572	0.371	0.224	0.528
occupancy_delta_only	0.430	0.445	0.279	0.336

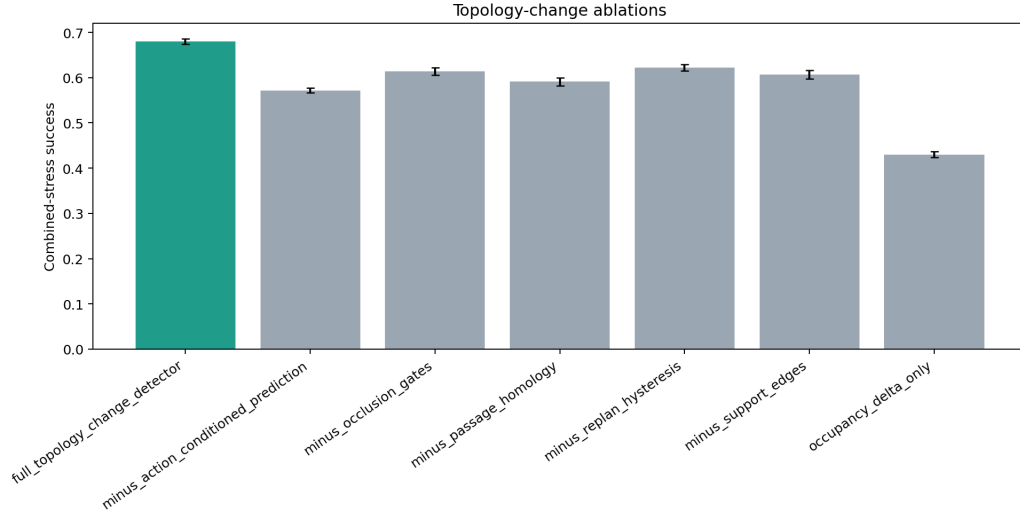


Figure 5: Ablation success under combined stress. No removed-component variant matches the full method.

10 LIMITATIONS AND TERMINAL DECISION

This is not a submission-ready robotics paper. The benchmark is local, the detector is an executable proxy rather than a trained model, and the topology-change families are not calibrated from hardware logs. The paper also lacks external benchmark results, rollout videos, and a full manual related-work synthesis.

The correct terminal decision after the 2026-06-15 continuation rerun is **STRONG-REVISE**. The mechanism should continue only if the next version adds real robot or accepted benchmark validation against dynamic scene graph, topological SLAM/TAMP, robust replanning, and learned affordance baselines.

REFERENCES

- 3D Dynamic Scene Graphs. 3d dynamic scene graphs: Actionable spatial perception with places, objects, and humans. arXiv:2002.06289, 2020. URL <https://arxiv.org/abs/2002.06289>.
- Iro Armeni, Zhi-Yang He, JunYoung Gwak, Amir R. Zamir, Martin Fischer, Jitendra Malik, and Silvio Savarese. 3d scene graph: A structure for unified semantics, 3d space, and camera. arXiv:1910.02527, 2019. URL <https://arxiv.org/abs/1910.02527>.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics

Table 3: Pairwise combined-stress success differences against the proposed method.

Baseline	Diff	CI	Wins
graph_neural_change_classifier	0.145	0.009	7
learned_affordance_map	0.234	0.013	7
occupancy_delta_replanner	0.266	0.011	7
oracle_topology_planner	-0.152	0.009	0
risk_aware_robust_replanner	0.204	0.009	7
static_scene_graph_planner	0.323	0.006	7
topological_slam_tamp	0.114	0.006	7
uncertainty_triggered_replanner	0.245	0.010	7

transformer for real-world control at scale. arXiv:2212.06817, 2023. URL <https://arxiv.org/abs/2212.06817>.

Nathan Hughes, Yun Chang, and Luca Carlone. Hydra: A real-time spatial perception system for 3d scene graph construction and optimization. arXiv:2201.13360, 2022. URL <https://arxiv.org/abs/2201.13360>.

Antoni Rosinol, Marcus Abate, Yun Chang, and Luca Carlone. Kimera: From slam to spatial perception with 3d dynamic scene graphs. arXiv:2101.06894, 2020. URL <https://arxiv.org/abs/2101.06894>.

Siddharth Srivastava, Eugene Fang, Lorenzo Riano, Rohan Chitnis, Stuart Russell, and Pieter Abbeel. Combined task and motion planning through an extensible planner-independent interface layer. ICRA, 2014. URL <https://ieeexplore.ieee.org/document/6907404>.